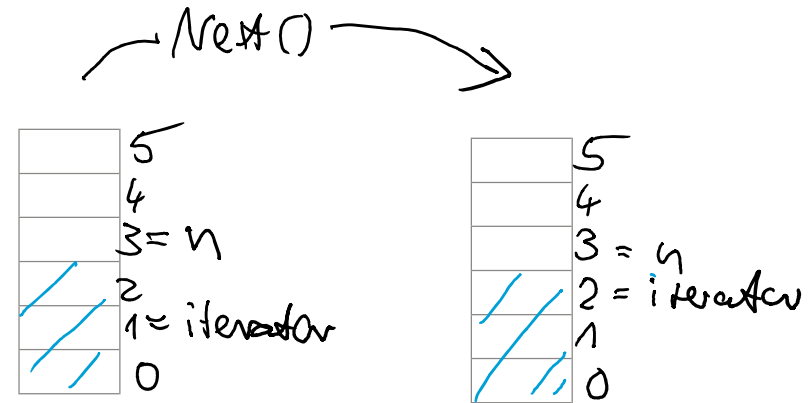


Implementierung Tabelle

// nächster Record wird zum aktuellen Record

```
template <class Tr>
boolean Table<Tr>::Next()
{
    if (n>0 && iterator>=0 && iterator<n-1)
    {
        iterator++;
        return true;
    }
    iterator=-1;
    return false;
}
```



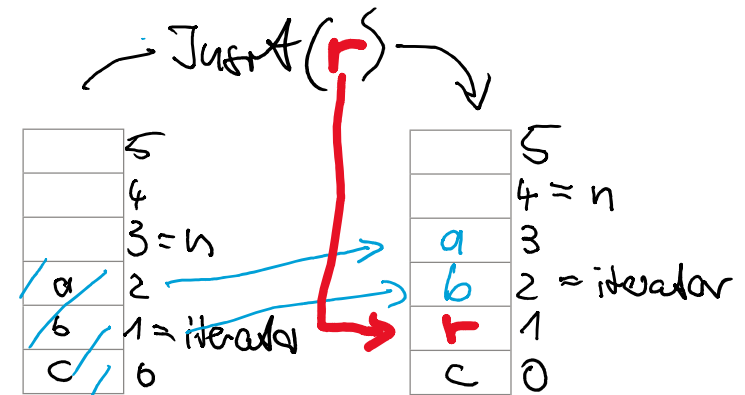
Sonderfälle :

1. Liste leer : return false
2. Iterator nicht positioniert; return false
3. Iterator = n-1: return false

Implementierung Tabelle

fügt neuen Record vor dem aktuellen Record ein

```
template <class Tr>
boolean Table<Tr>::Insert(Tr &r)
{
    if (iterator>=0 && n<max)
    {
        for (int i=n-1; i>=iterator; i--)
            a[i+1]=a[i]; // umspeichern einschließlich Iterator-Index
        a[iterator]=r;
        n++; // Anzahl Records aktualisieren
        iterator++; // Iterator zeigt auf den alten Record
        return true;
    }
    else return false;
}
```



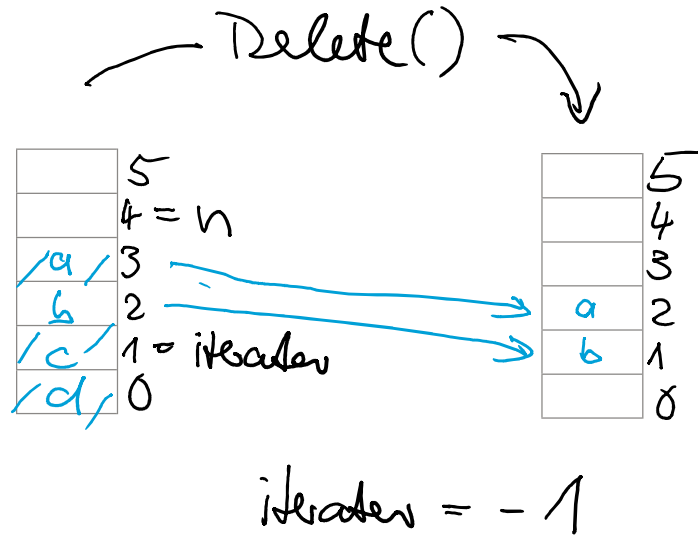
Sonderfälle:

1. Liste leer $\Rightarrow$ return false
2. Liste voll $\Rightarrow$ return false
3. Iterator nicht positioniert $\Rightarrow$ return false

Implementierung Tabelle

löscht aktuellen Record

```
template <class Tr>
boolean Table<Tr>::Delete()
{
    if (iterator<0) return false;
    for (int i=iterator; i<n-1; i++) a[i]=a[i+1];
    n--;
    iterator=-1;
    return true;
}
```



Sonderfälle

1. iterator nicht positioniert $\Rightarrow$ return false
2. Tabelle leer $\Rightarrow$ return false